

Engines of AI

From Data to Solutions

ETH Zurich

28 August 2026

Ankita Ghosh

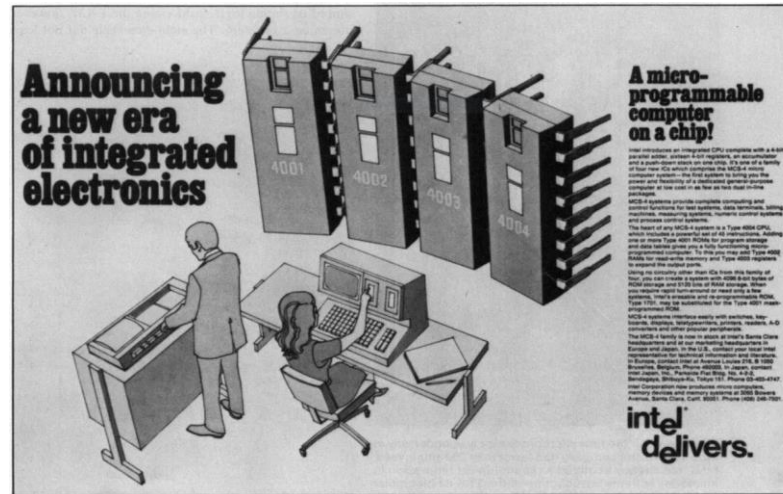


Timeline: Fifty years of Computing

1971

The first CPU

Intel 4004: one chip that could follow instructions



Intel 4004 advertisement in Electronic News magazine from 1971



Basicom 141-PF calculator: first implementation of the 4004 chipset



IBM 1620 Data Processing System: punch card mechanism, late 1960s



A MITS Altair 8800 (left) with an ASR 33 teletype (right), 1975



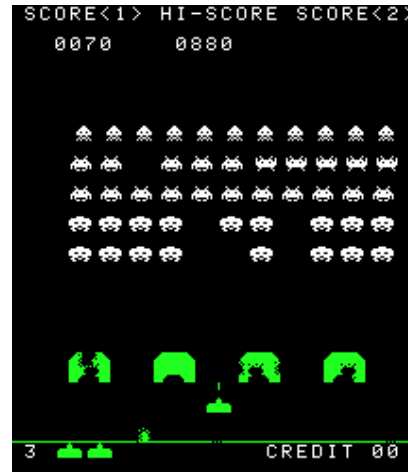
Asteroids arcade game, vector graphics, 1979

Timeline: Fifty years of Computing

1971

The first CPU

Intel 4004: one chip that could follow instructions



Space Invaders arcade game, raster graphics, 1979

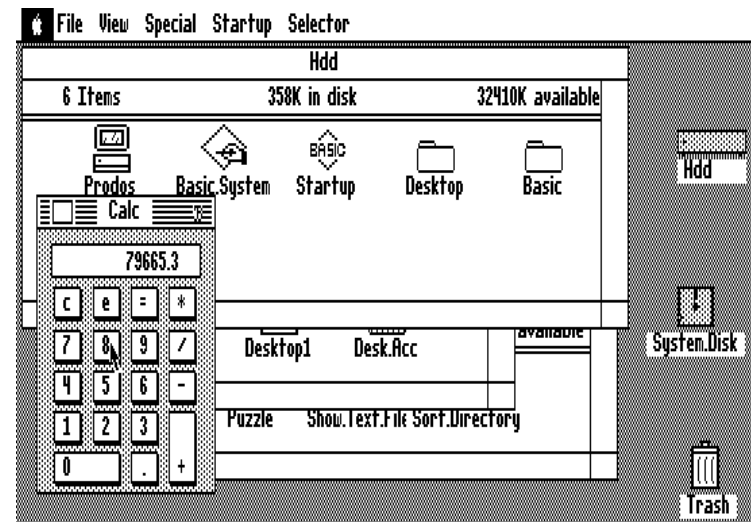


Pac-man arcade game, raster graphics, 1980

1980s

Rasterization and 3D Graphics

Pictures have to be drawn, pixel by pixel



Apple II Computer GUI, raster graphics, 1980s

Timeline: Fifty years of Computing

1971

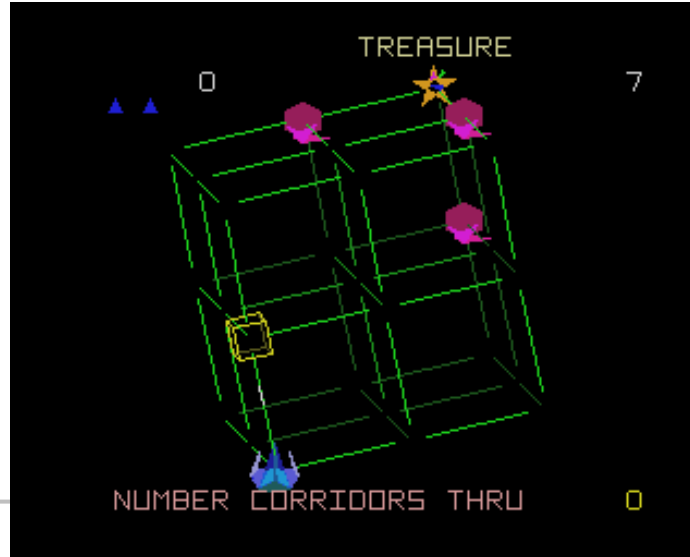
The first CPU

Intel 4004: one chip that could follow instructions

1980s

Rasterization and 3D Graphics

Pictures have to be drawn, pixel by pixel



Cube Quest, 3D graphics rendering, 1983



I, Robot, 3D graphics rendering, 1984

Timeline: Fifty years of Computing

1971

The first CPU

Intel 4004: one chip that could follow instructions

1999

The GPU, for gaming

Hardware built to draw millions of pixels fast



Nvidia GeForce 256, 1999

1980s

Rasterization and 3D Graphics

Pictures have to be drawn, pixel by pixel

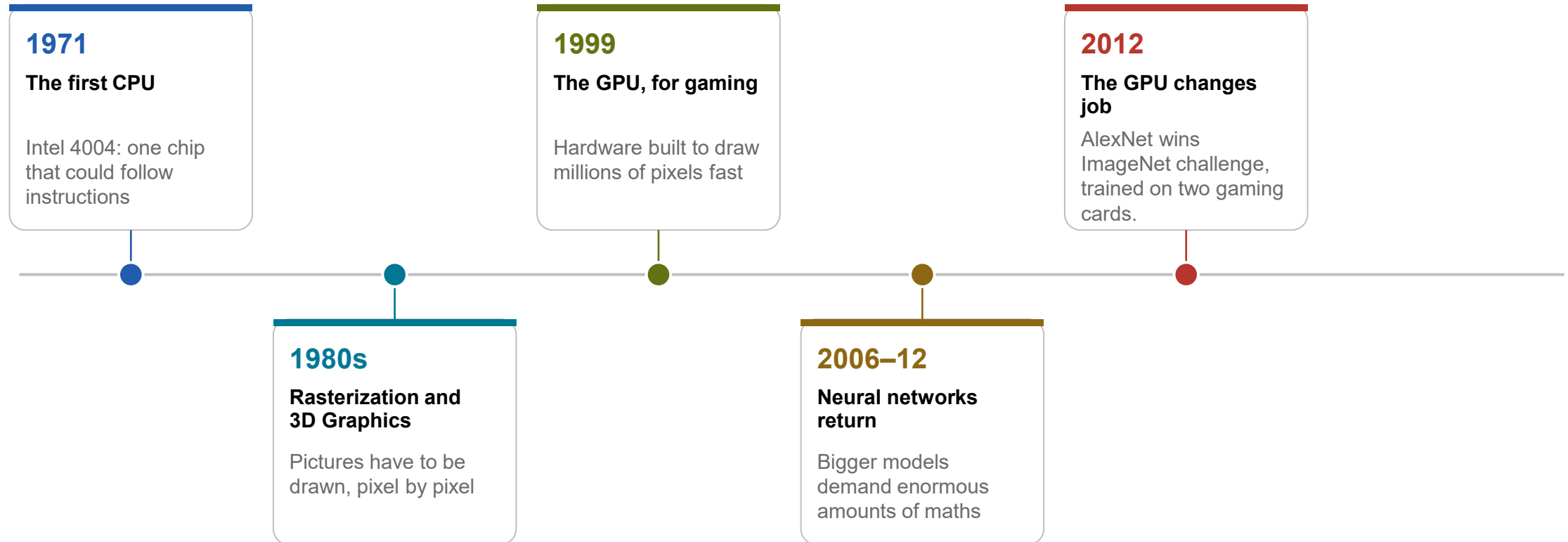


Quake III Arena, video game, 1999



Dagorath Moor Zoological Gardens, video game, 1999

Timeline: Fifty years of Computing



Timeline: Fifty years of Computing

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

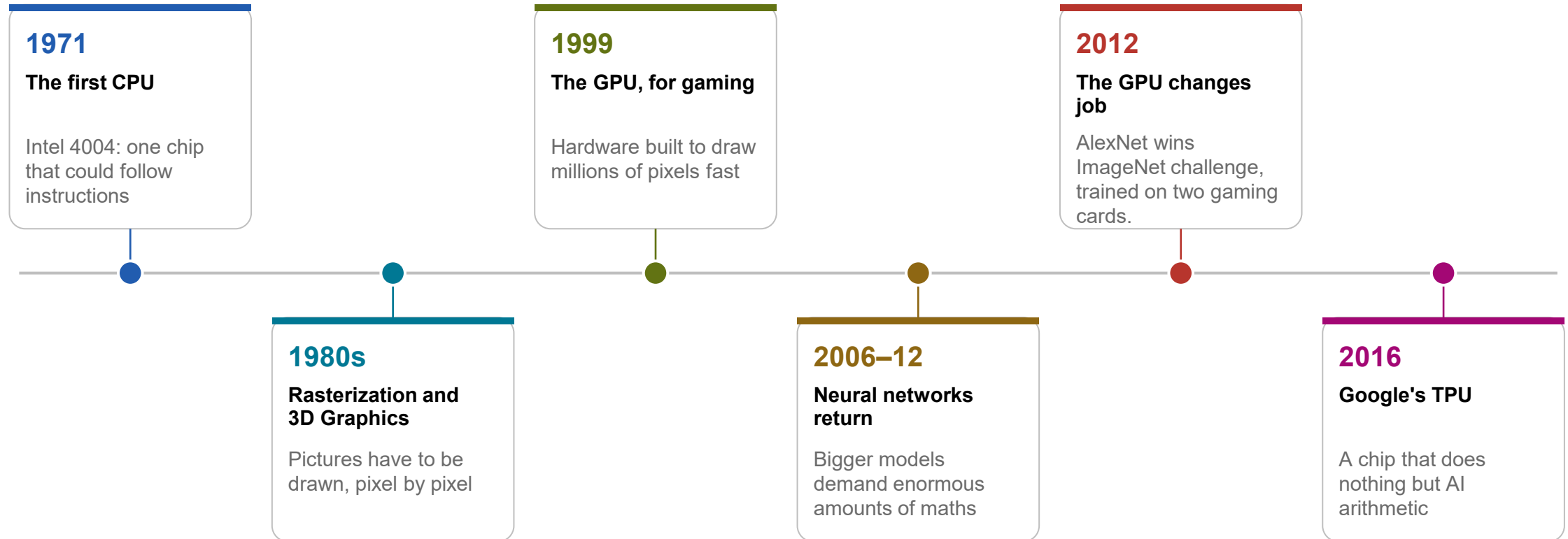
Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105.

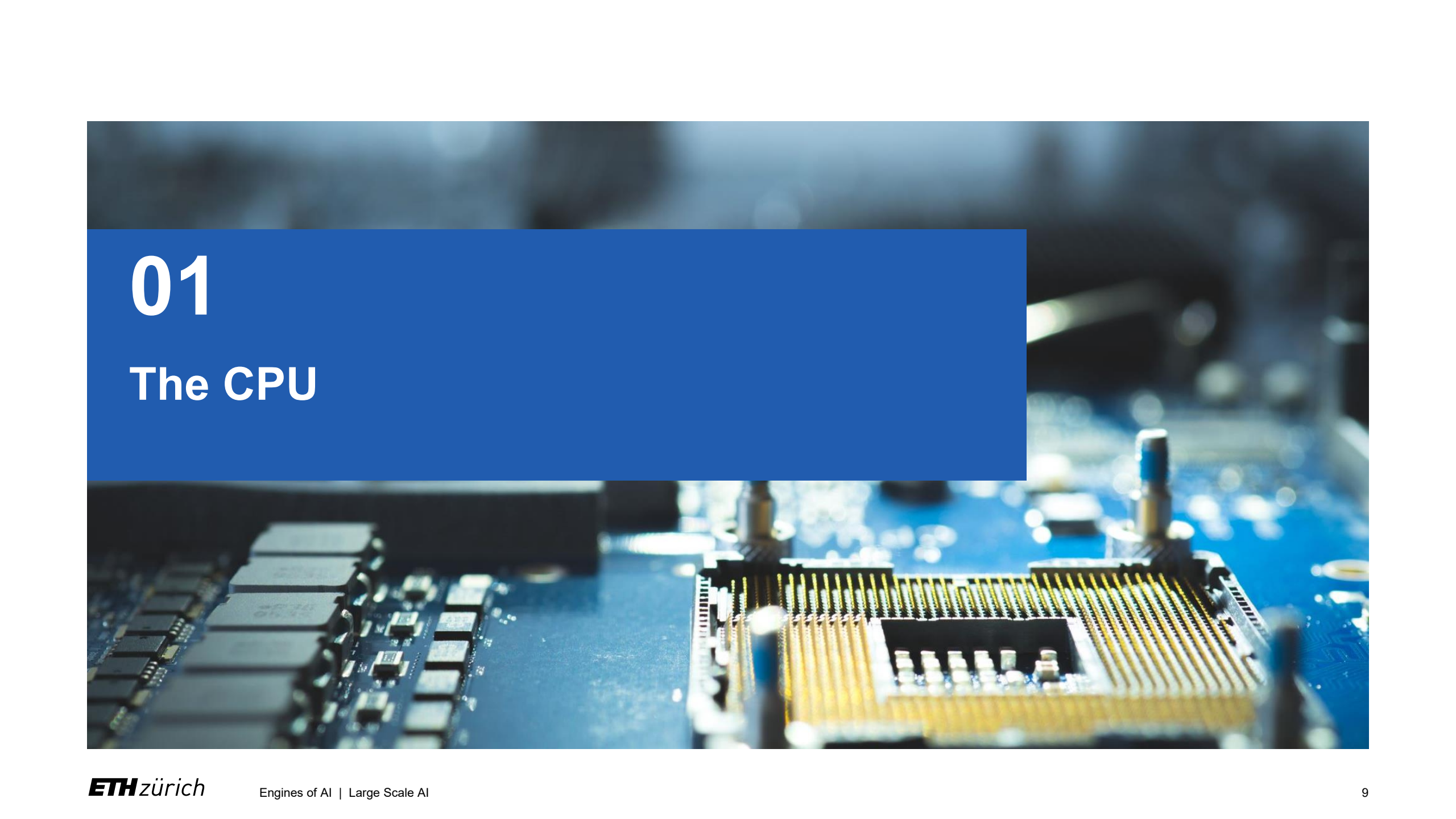
2012

The GPU changes job

AlexNet wins ImageNet challenge, trained on two gaming cards.

Timeline: Fifty years of Computing



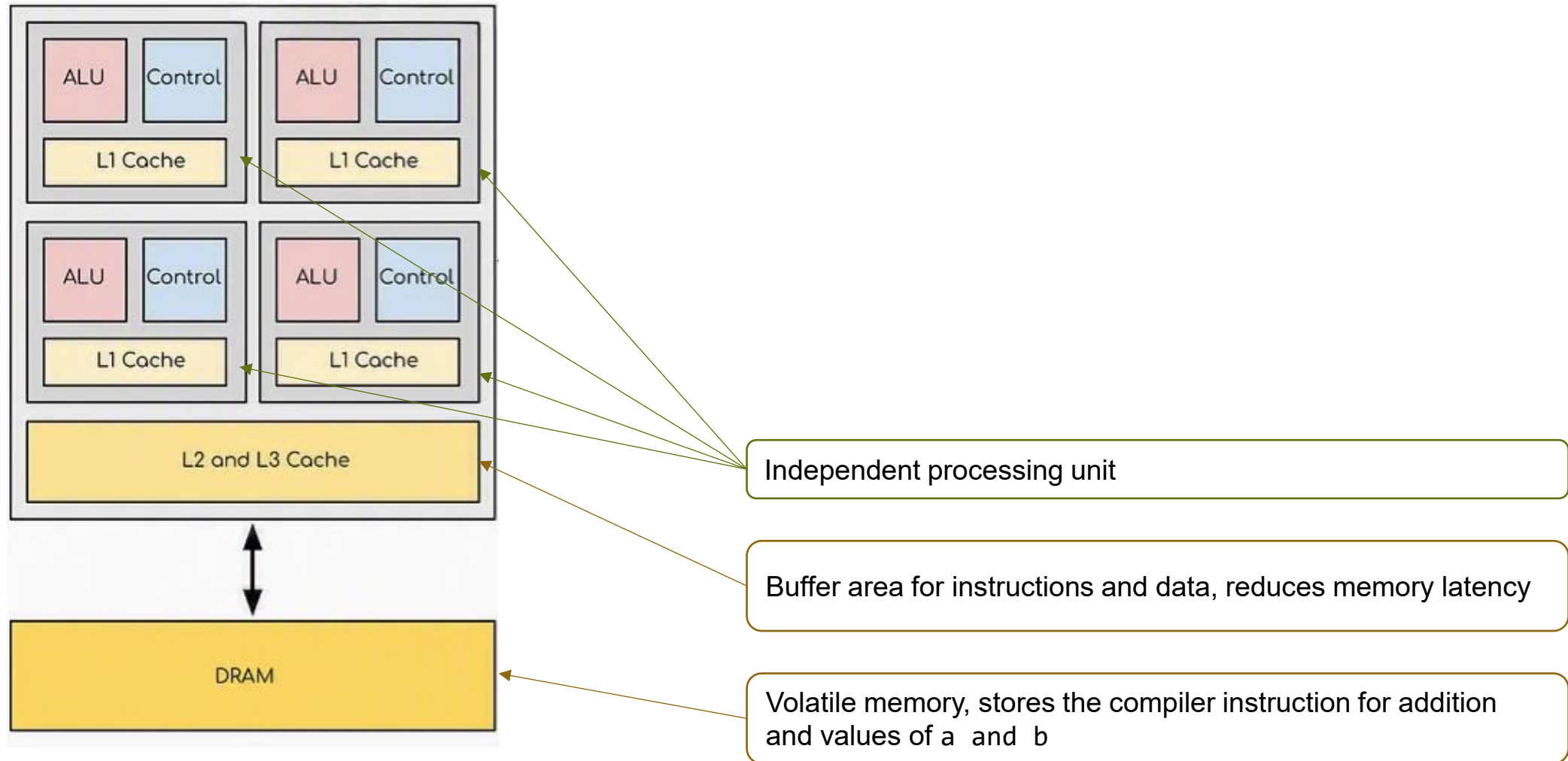


01

The CPU

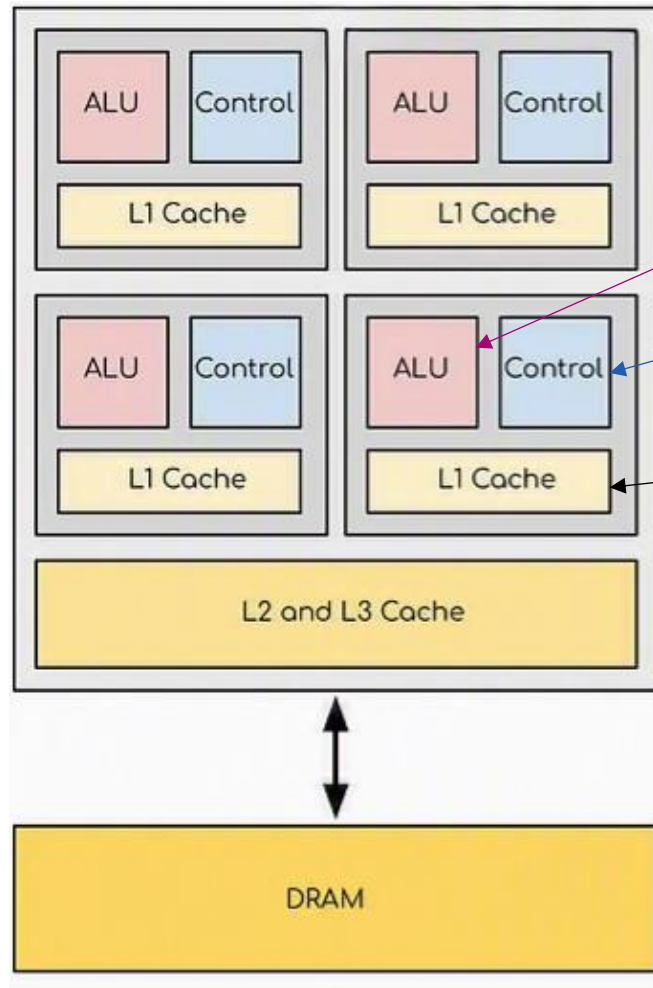
Inside a Central Processing Unit (CPU)

Let's consider a line of code: $c = a \times b$



Inside a Central Processing Unit (CPU)

Let's consider a line of code: $c = a \times b$



Physically executes the arithmetic ($a \times b$) or logical operation

Decodes instruction (multiply) and directs data flow (a and b)

Specific core assigned to this task pulls the instruction and variables into its ultra-fast, private memory for immediate access

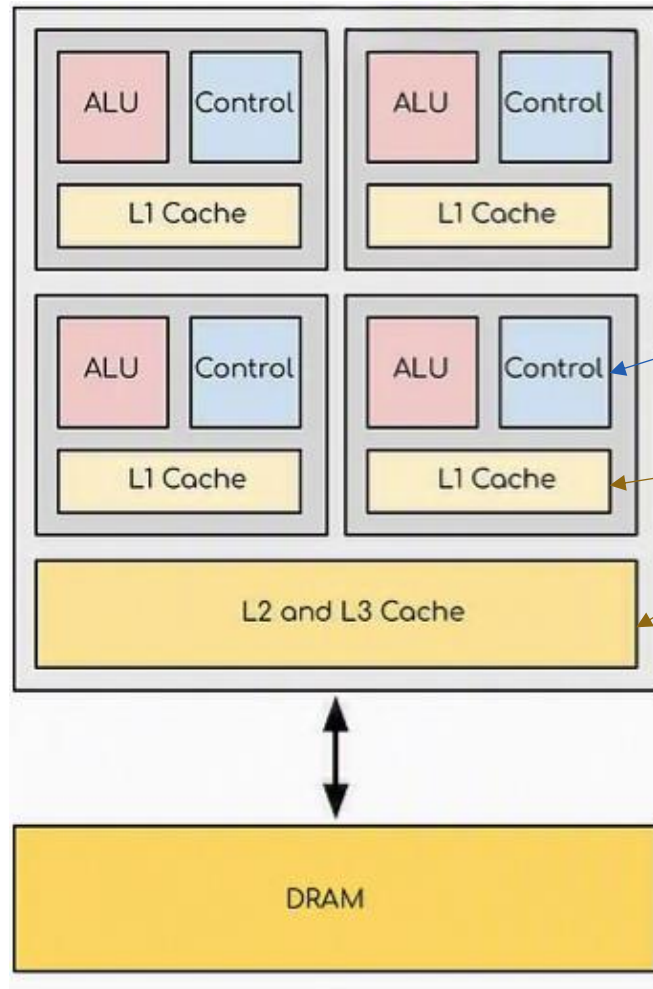
Independent processing unit

Buffer area for instructions and data, reduces memory latency

Volatile memory, stores the compiler instruction for addition and values of a and b

Inside a Central Processing Unit (CPU)

Let's consider a line of code: $c = a \times b$



Physically executes the arithmetic ($a \times b$) or logical operation

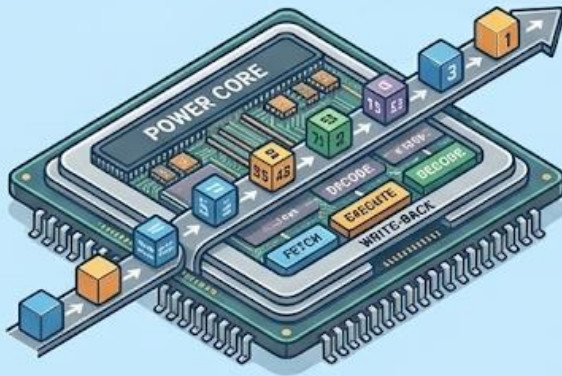
Directs data flow of computed c

Result c written back down to DRAM through L1, L2, L3 caches.

CPU Compute and Usage

Architecture

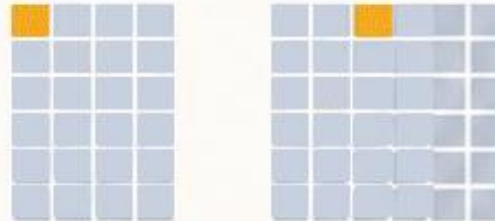
Few Highly Powerful Cores



Designed for Sequential processing and complex decision-making

Compute

Scalar + SIMD
(latency-optimized)



Optimized for low-latency, control-flow-heavy workloads

Application

General-Purpose Workloads



Excel at Running Operating Systems, Managing Databases, & General Productivity

If CPU is so powerful, why can it not handle rendering graphics?

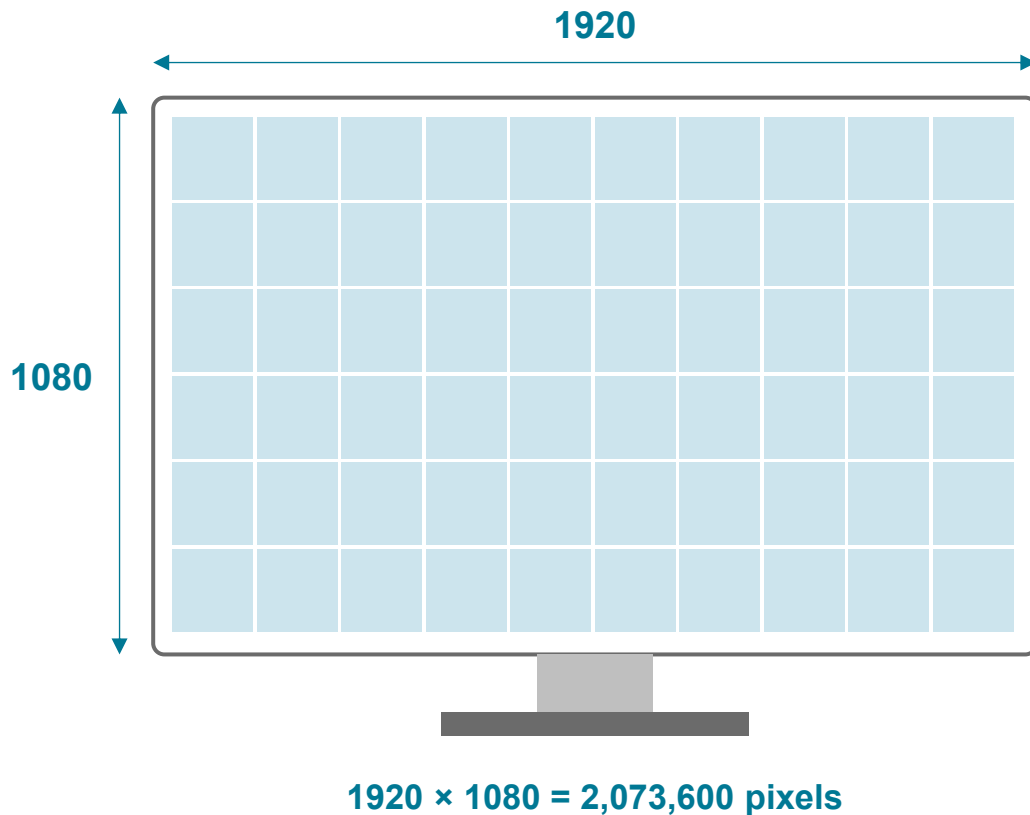
The background of the slide features a 3D rendering of numerous cubes. These cubes are arranged in a somewhat haphazard, stacked manner, creating a sense of depth and volume. The cubes are rendered with soft lighting, showing subtle gradients of purple and pink on their faces. The overall background is a smooth, light purple gradient that complements the cube colors.

02

3D Graphics and Rasterization

Raster Graphics: a screen full of pixels

Raster graphics are digital images made up of a rectangular grid of squares called pixels.



2.1 million

pixels on an ordinary screen

60×

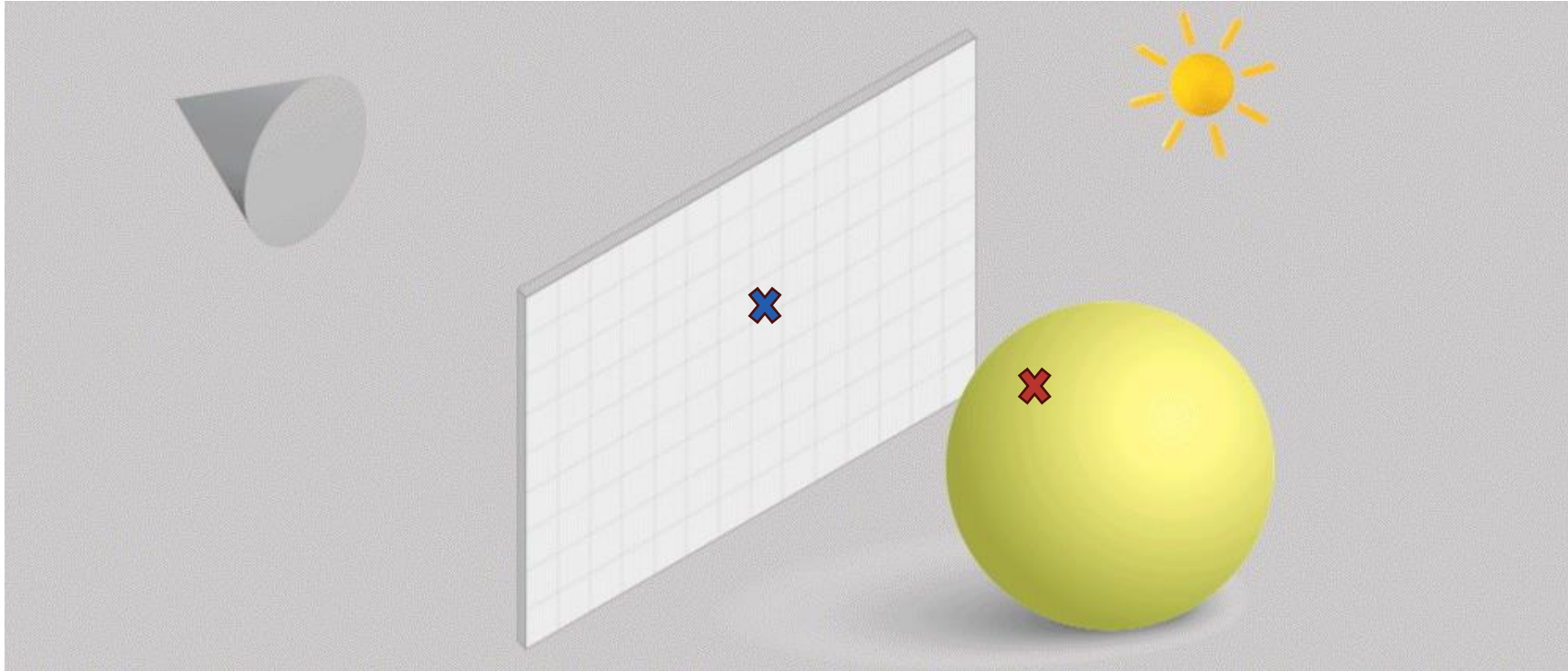
the whole screen is redrawn, every second

124 million

pixel updates every single second

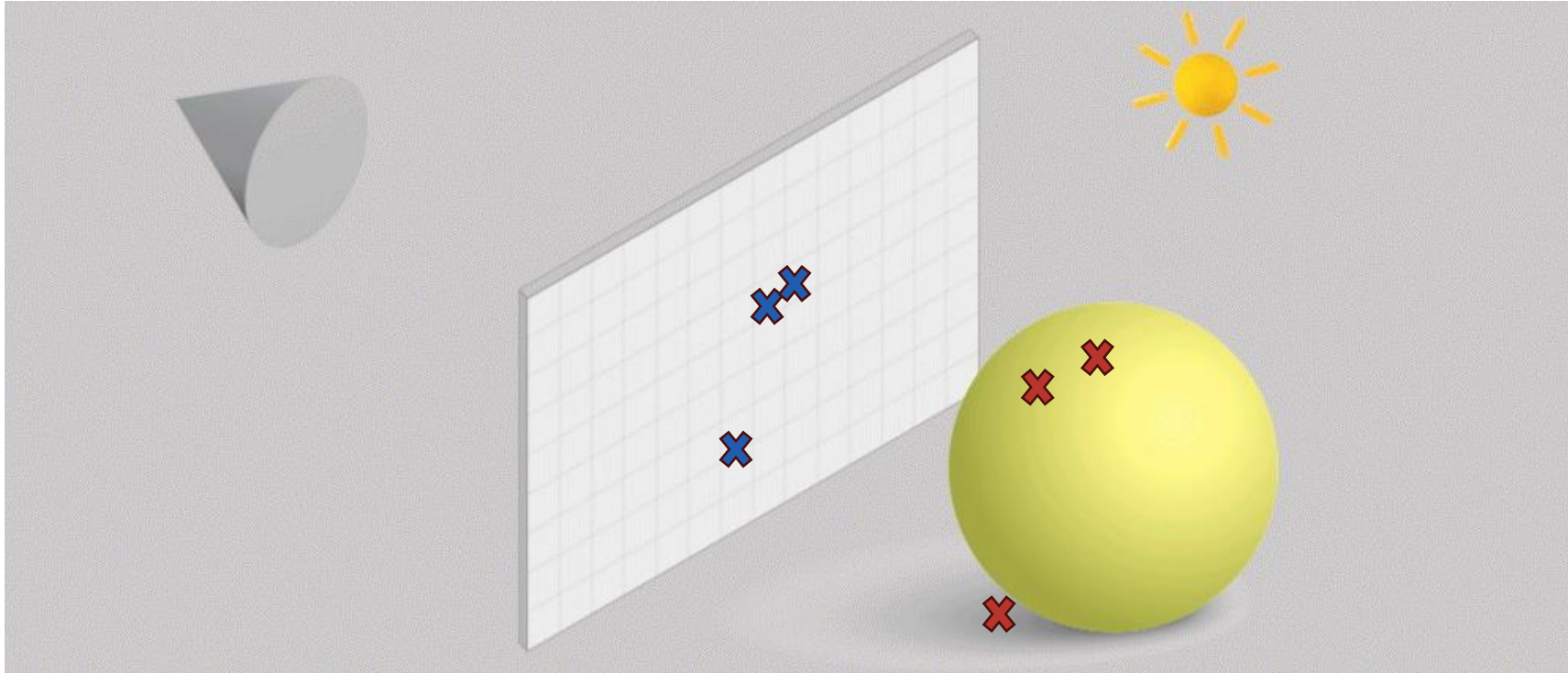
Every second, your computer processor needs to make millions of updates.

3D Graphics and Rendering



Pixel value (P) = Transformations (T) x Vertex value (V)

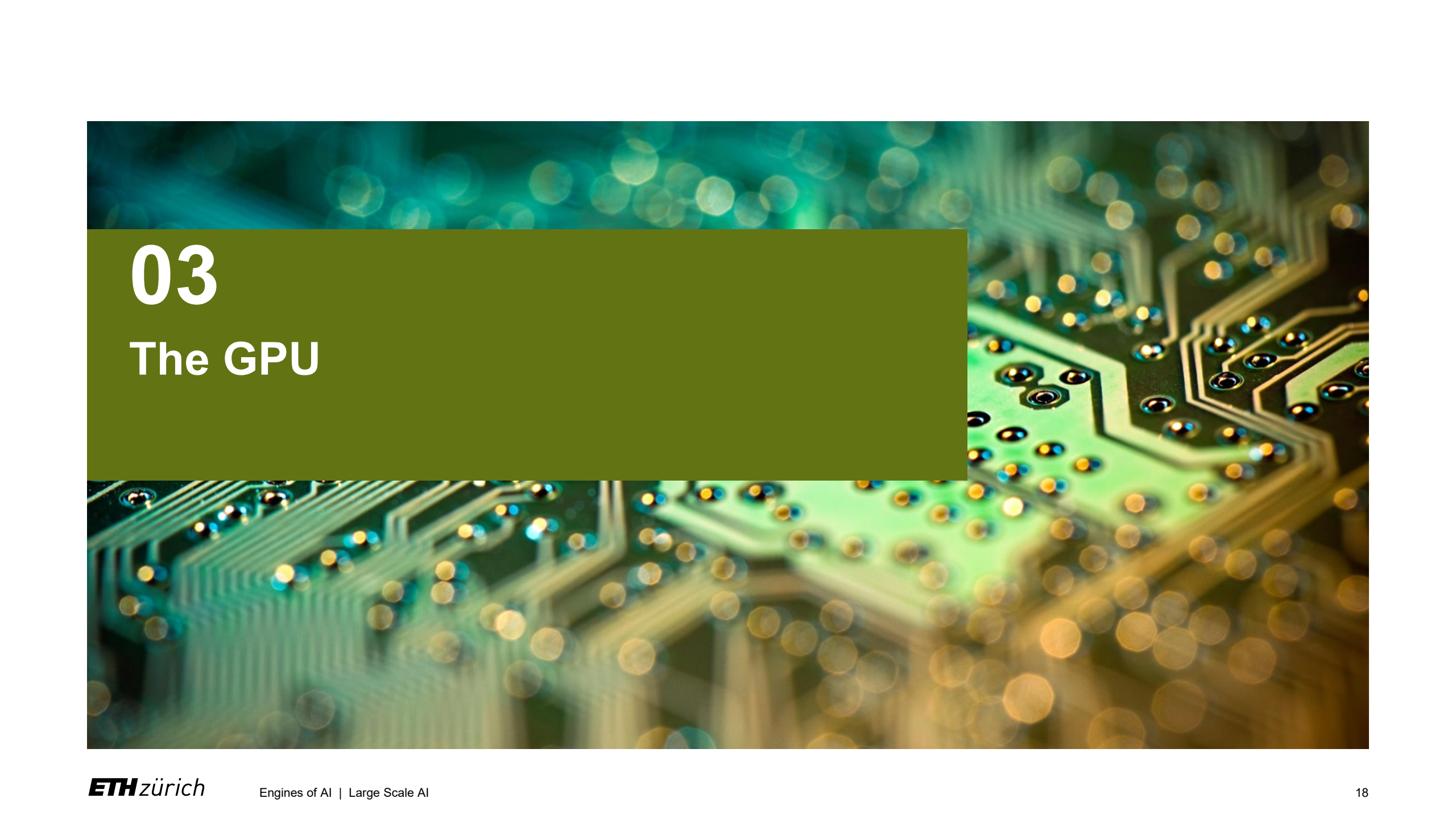
3D Graphics and Rendering



$$\begin{bmatrix} P_1 \\ P_2 \\ \dots \\ P_n \end{bmatrix} = \begin{bmatrix} T_1 & T_2 & \dots & T_n \end{bmatrix} \times \begin{bmatrix} V_1 \\ V_2 \\ \dots \\ V_n \end{bmatrix}$$

The same mathematics, millions of times over, with no sequential dependency → **matrix operations**

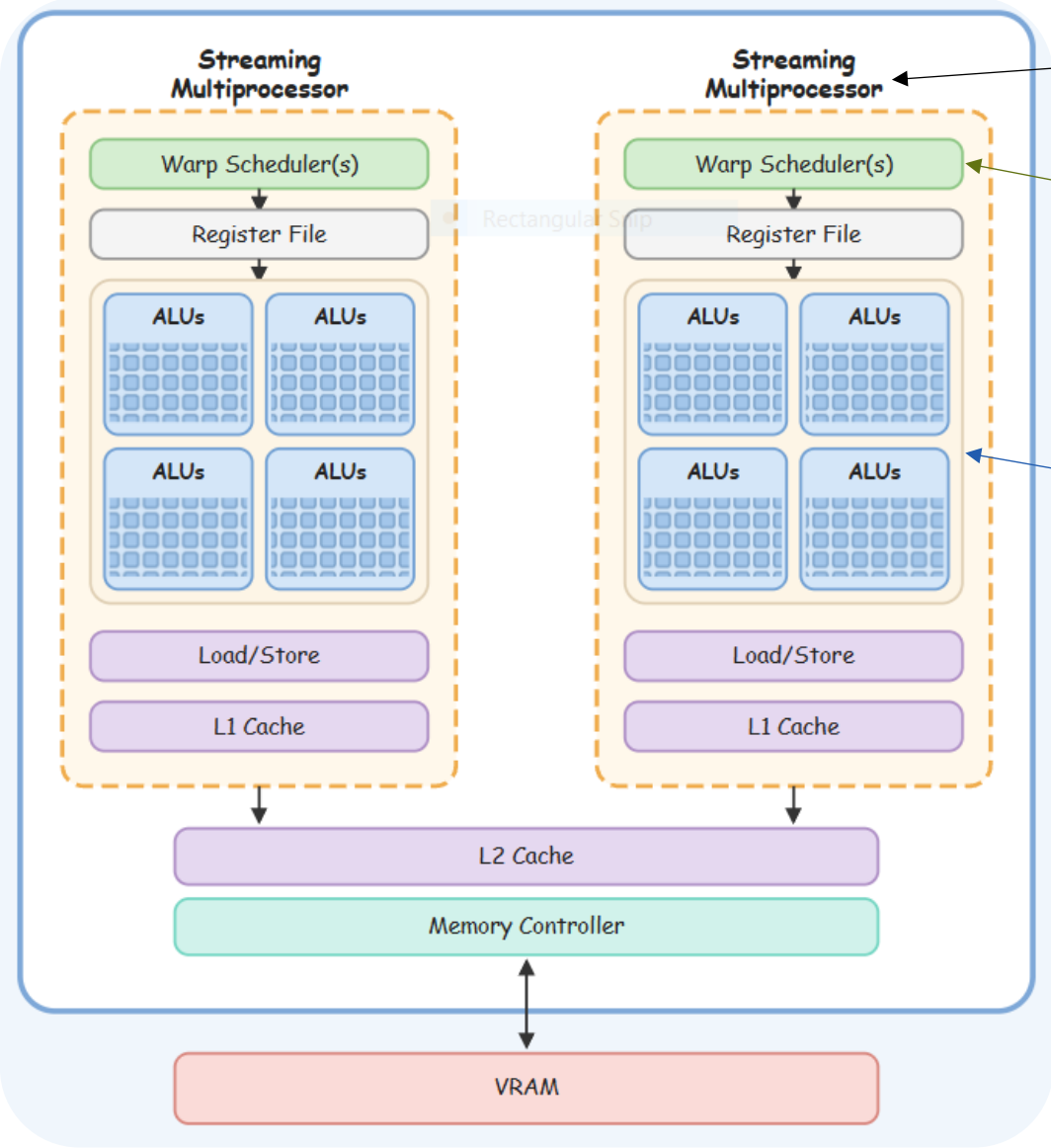
A CPU takes a long long time to execute this.



03

The GPU

Inside a Graphics Processing Unit



Core parallel processing block of the GPU

Organizes and dispatches instructions to groups of parallel instructions

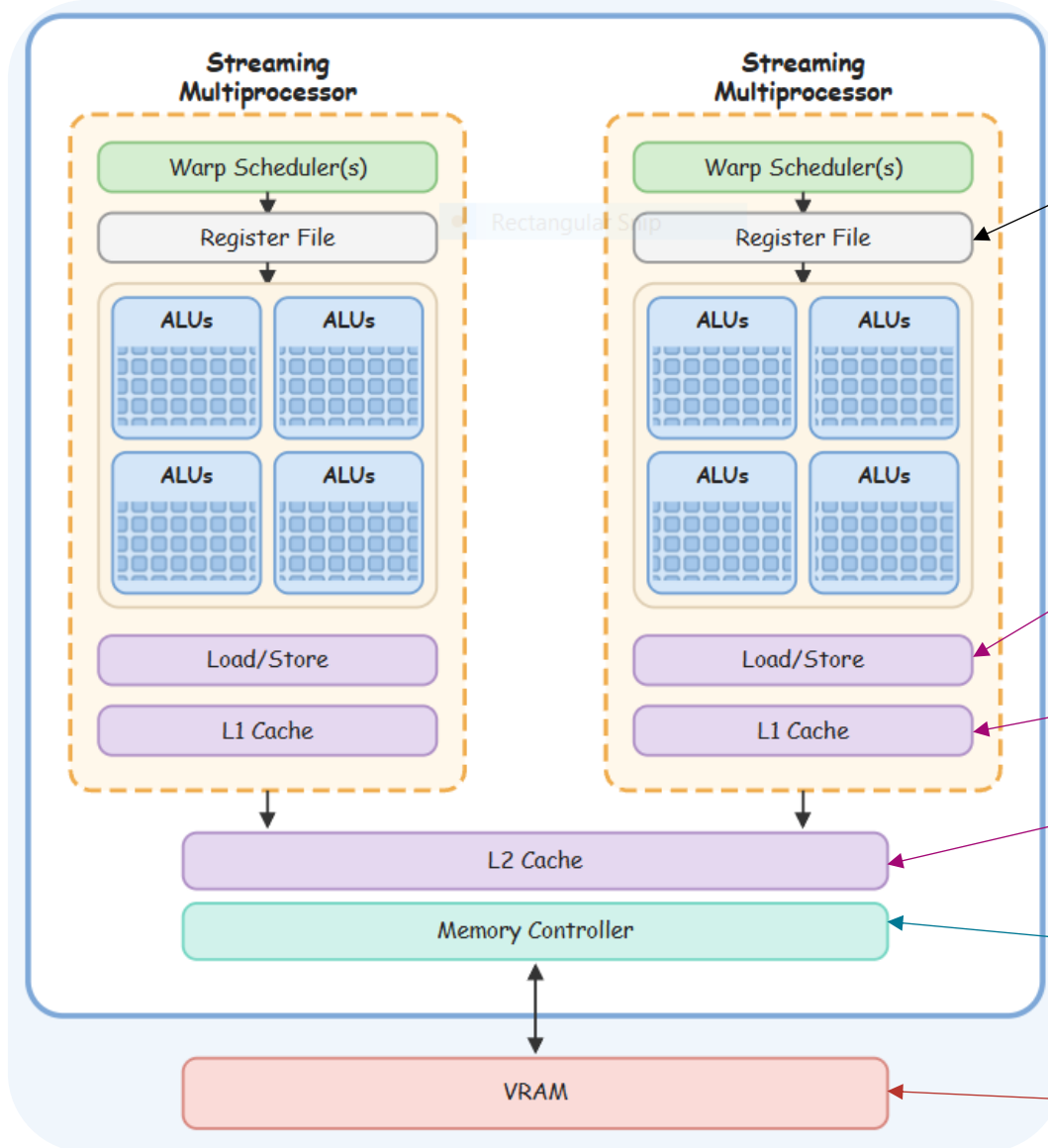
Scheduler says “Multiply”

Massive grids for executing the mathematical and logical operations.

All ALUs perform multiplication operation

$$\textcolor{blue}{<P>} = \textcolor{black}{<T} \times \textcolor{red}{V} \textcolor{black}{>}$$

Inside a Graphics Processing Unit



The closest on-chip memory for extremely fast execution.

Dedicated hardware units that handle reading and writing of data.

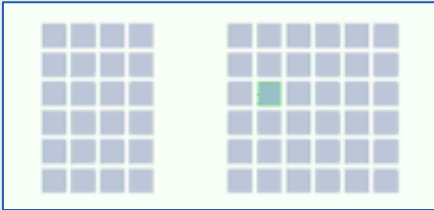
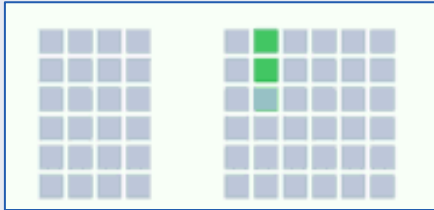
Local to each Streaming Multiprocessor.

Shared layer that acts as a central data buffer.

Interface that manages data requests between the GPU's internal caches and the external memory.

High-capacity, off-chip memory that stores all the data.

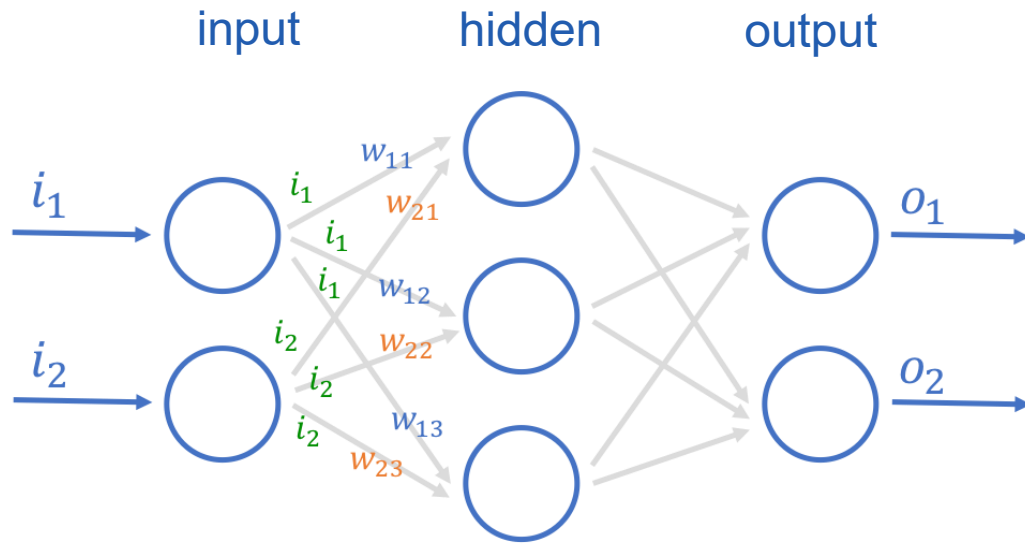
CPU versus GPU

	CPU	GPU
Processors	6 – 24, each very powerful cores	5,000 – 20,000, each simple ALUs
Good at	Decisions, order, control	The same maths on a lot of data
Memory	Large, moderately fast	Smaller, extraordinarily fast
Designed for	Latency — one job, finished fast	Throughput — many jobs at once
Weak at	Bulk arithmetic	Branching, unpredictable logic
Compute		
Picture it as	The head chef: can prep, cook, customize	Fast-food crew: Hundred line cooks output perfect burgers simultaneously on hearing “flip”.

04

Neural Networks Resurges

What a neural network actually computes



$$\text{next layer} = f_{\text{activation}} (\text{Weights} \times \text{previous layer} + \text{bias})$$

So for first hidden neuron: $h_1 = (w_{11} \times i_1) + (w_{21} \times i_2) + b$

$$\begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} \cdot \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} = \begin{bmatrix} (w_{11} \times i_1) + (w_{21} \times i_2) \\ (w_{12} \times i_1) + (w_{22} \times i_2) \\ (w_{13} \times i_1) + (w_{23} \times i_2) \end{bmatrix} + b$$

Looks familiar to something?

Neural networks also compute matrix operations like graphics rendering!

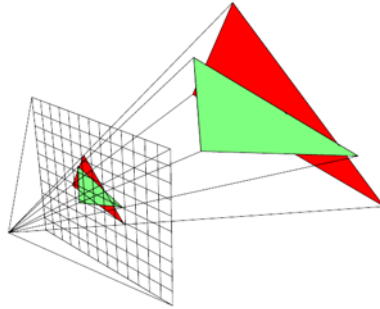
$$\text{Pixel value (P)} = \text{Transformations (T)} \times \text{Vertex value (V)}$$

$$\begin{bmatrix} P_1 \\ P_2 \\ \dots \\ P_n \end{bmatrix} = \begin{bmatrix} T_1 & T_2 & \dots & T_n \end{bmatrix} \times \begin{bmatrix} V_1 \\ V_2 \\ \dots \\ V_n \end{bmatrix}$$

Two problems, one shape of mathematics

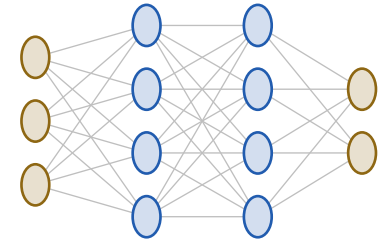
Drawing a picture

- Millions of points to place
- The same transform for each one
- None of them waits for another



Training a network

- Millions of weights to multiply
- The same multiply for each one
- None of them waits for another



**Both are: multiply big grids of numbers, over and over, independently.
A chip built for one is a chip built for the other.**

2012: The Moment It Clicked

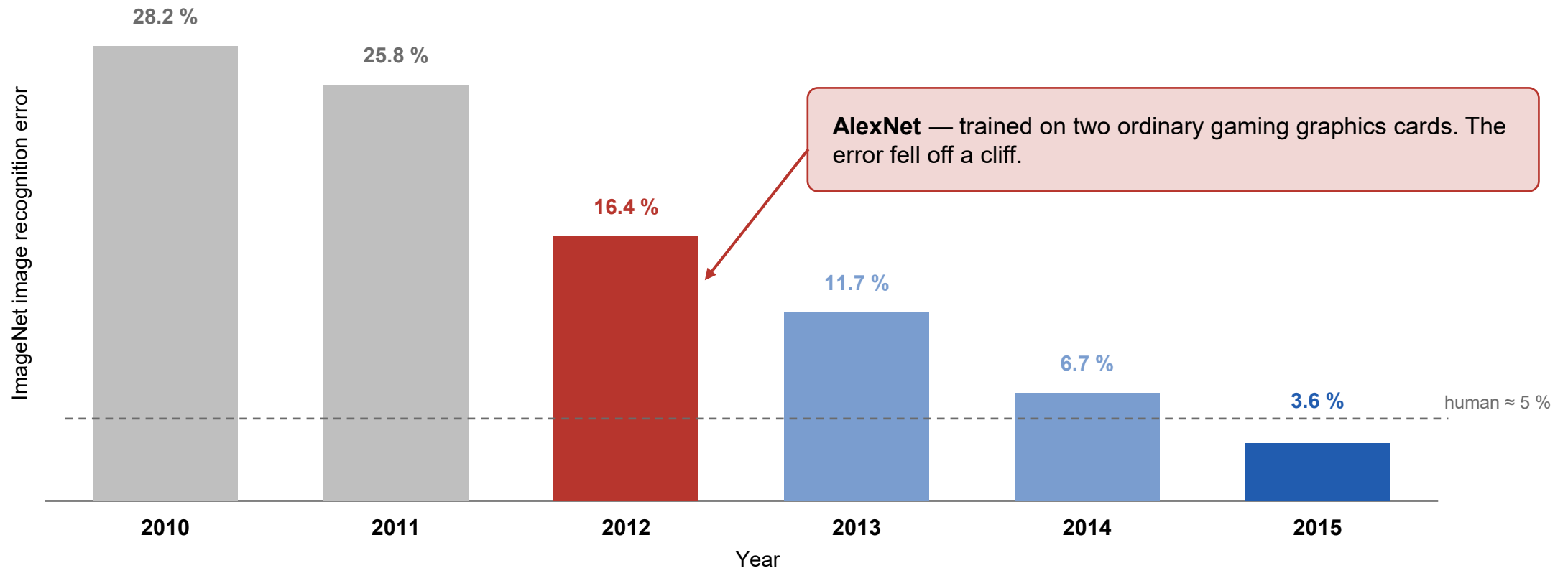


ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

Dataset: 1,000 distinct object categories with roughly 1.2 million training images

Tasks: Include image classification, object localization, and object detection

Error rate: The fraction of test images where the model's highest-probability prediction does not match the correct target label

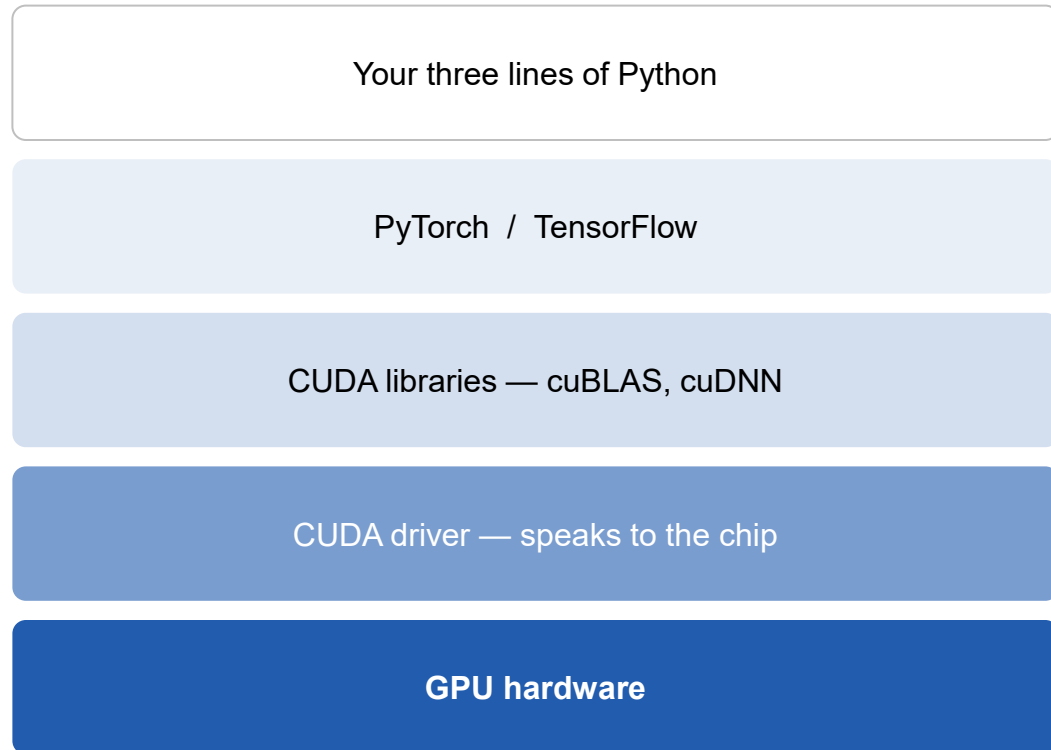


05

So why did NVIDIA take off?



CUDA: the moat was software



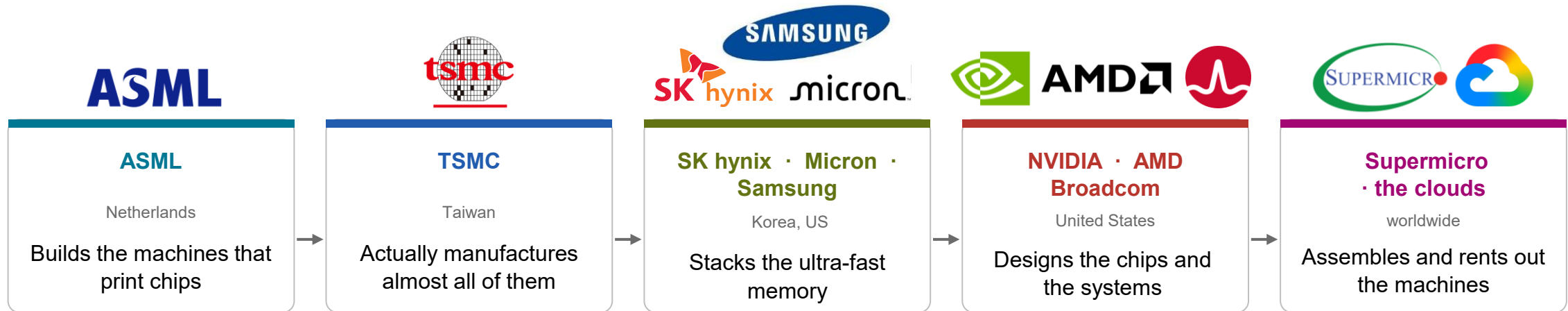
Every layer written — and given away — by NVIDIA, from 2006.

The unfair advantage

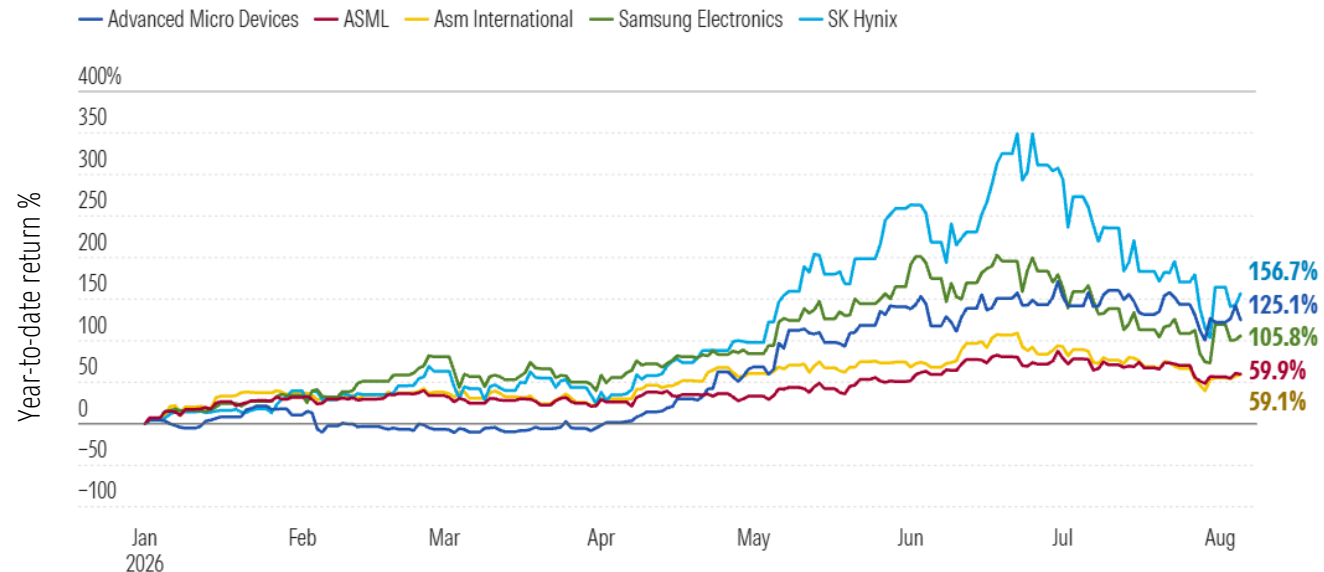
- 2006: CUDA ships — years before anyone needed it for AI.
- Before it, using a GPU for maths meant disguising your sums as graphics.
- NVIDIA gave it away and taught it in universities.
- By 2012, every researcher's code already assumed an NVIDIA card.

Rivals could match the silicon. Matching a decade of software — and everybody's habits — is far harder.

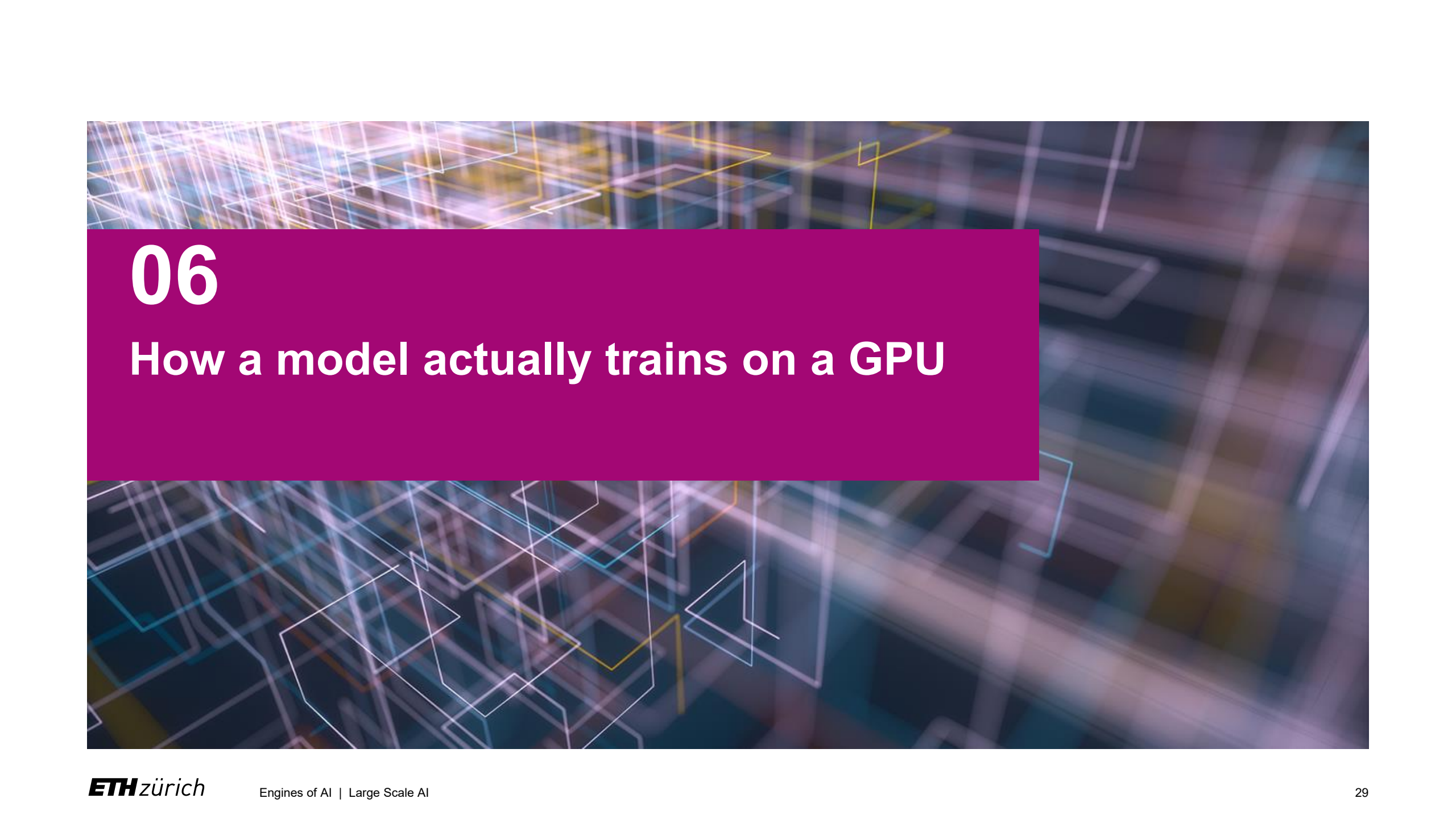
The AI Hardware Supply Chain



None of these companies trains AI models. They sell the equipment — and from 2023 their market values were re-rated sharply.



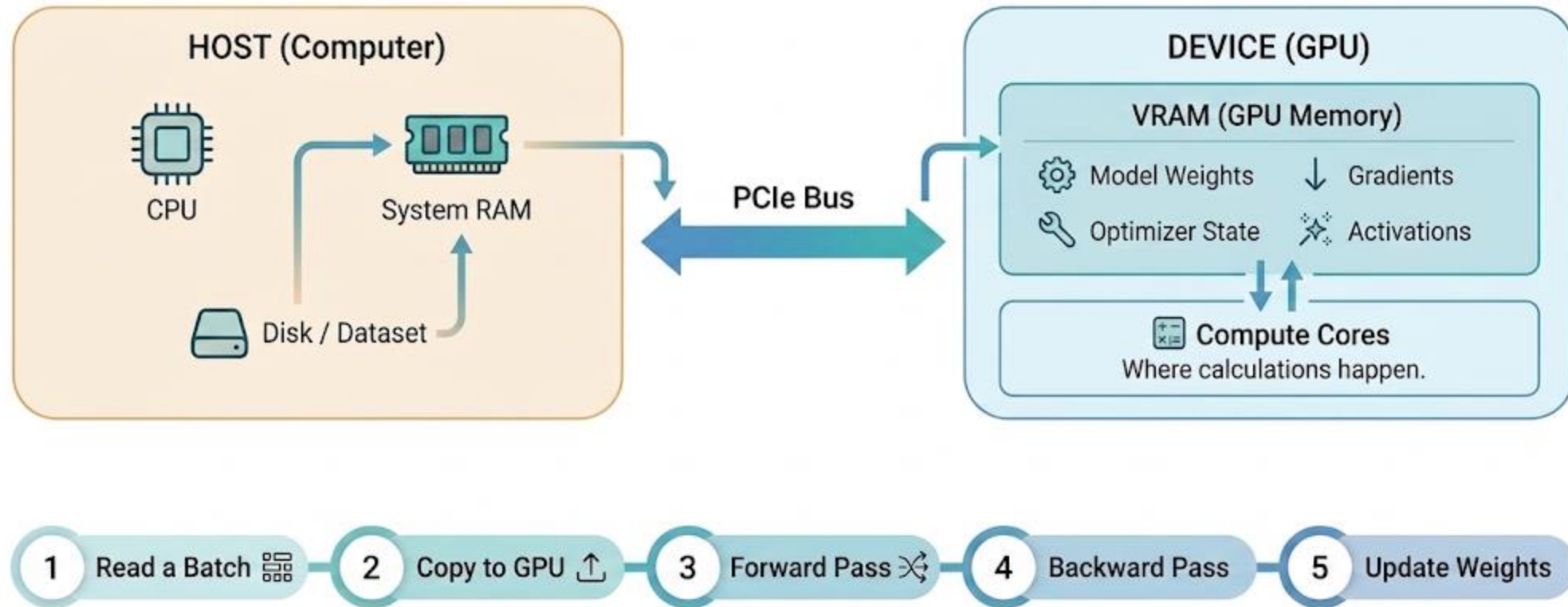
Source: Morningstar. Data as of Aug. 5, 2026. [Download CSV](#).



06

How a model actually trains on a GPU

Where everything lives during training



The commands you actually type

```
nvidia-smi
```

Ask the driver directly: which cards do I have, how hot are they, and how full is their memory?

```
torch.cuda.is_available()
```

Is there a working GPU and a matching driver? This returns False far more often than people expect.

```
model.to("cuda")
```

Copy every weight across the PCIe bus into GPU memory. From this moment they live there.

```
batch.to("cuda")
```

Copy this batch of data across too. Model and data have to be on the same side of the bridge.

```
torch.cuda.memory_allocated()
```

How much VRAM your tensors occupy right now — your early-warning light for running out.

```
torch.cuda.empty_cache()
```

Hand cached but unused blocks back. Useful, but it cannot conjure up memory you do not have.

How much VRAM do tensors occupy

One billion parameters, trained the ordinary way:



≈ 16 GB before you have processed a single image.

What “out of memory” really means

The card physically has no room left. Nothing is broken — the numbers simply do not fit.

The only three knobs

Smaller batch · smaller model · more GPUs.
Plus tricks: checkpointing, 8-bit optimizers.

When it breaks: what the error really means

CUDA out of memory. Tried to allocate 2.00 GiB

VRAM is full: weights, gradients, optimizer state and activations do not fit together.

Fix: Smaller batch, gradient accumulation or checkpointing, or more GPUs.

CUDA driver version is insufficient for CUDA runtime version

Your PyTorch was built for a newer CUDA than the installed driver understands.

Fix: Update the driver, or install the torch build that matches it.

Expected all tensors to be on the same device

Half your numbers are in system RAM and half in VRAM. The GPU cannot reach across.

Fix: Send both the model and the batch with `.to(device)`.

CUDA error: device-side assert triggered

A thread on the GPU hit an impossible value — usually a label outside the range of classes.

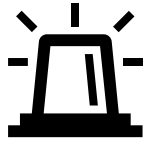
Fix: Re-run with `CUDA_LAUNCH_BLOCKING=1` to find the line that really failed.



07

Google's answer: the TPU

Google's problem: Arithmetic at Planetary Scale



The alarm, 2013

If every Android user spoke to their phone for three minutes a day, Google would have had to double its datacentres.



The decision

Do not build a faster general-purpose chip. Build one that does almost nothing but multiply matrices.

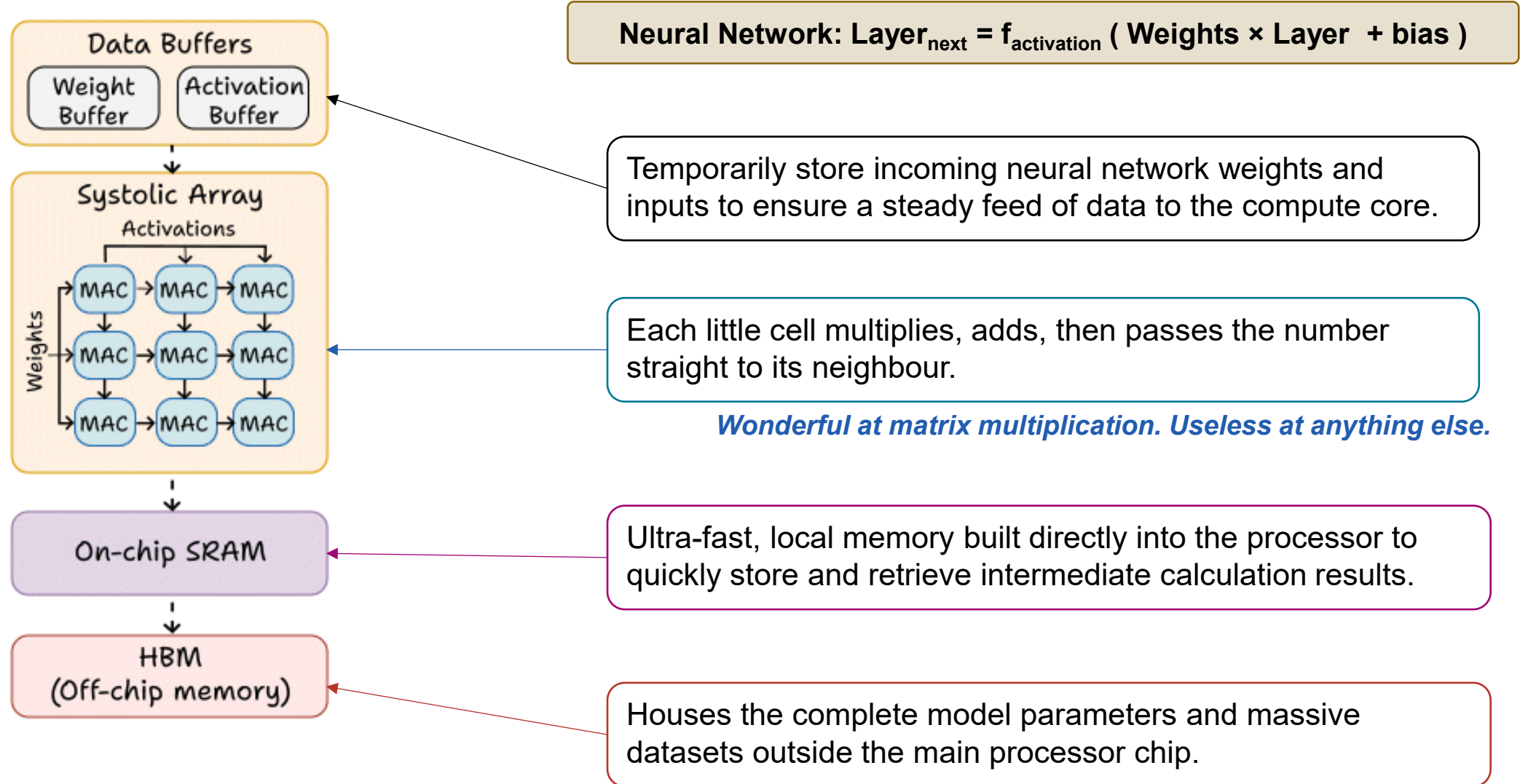


The result, 2016

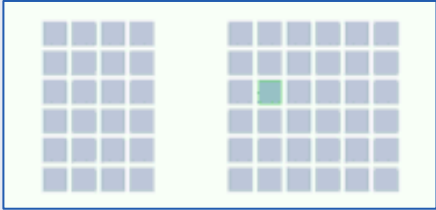
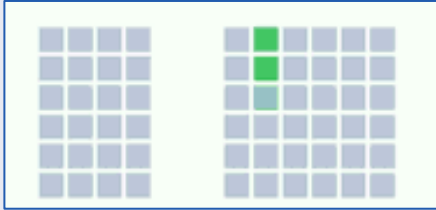
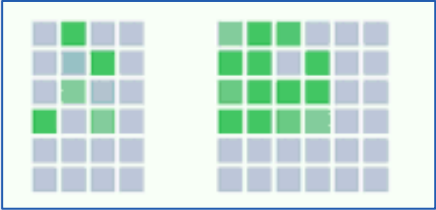
Running in datacentres by 2015, announced in 2016. Far more AI arithmetic per watt than anything general-purpose.

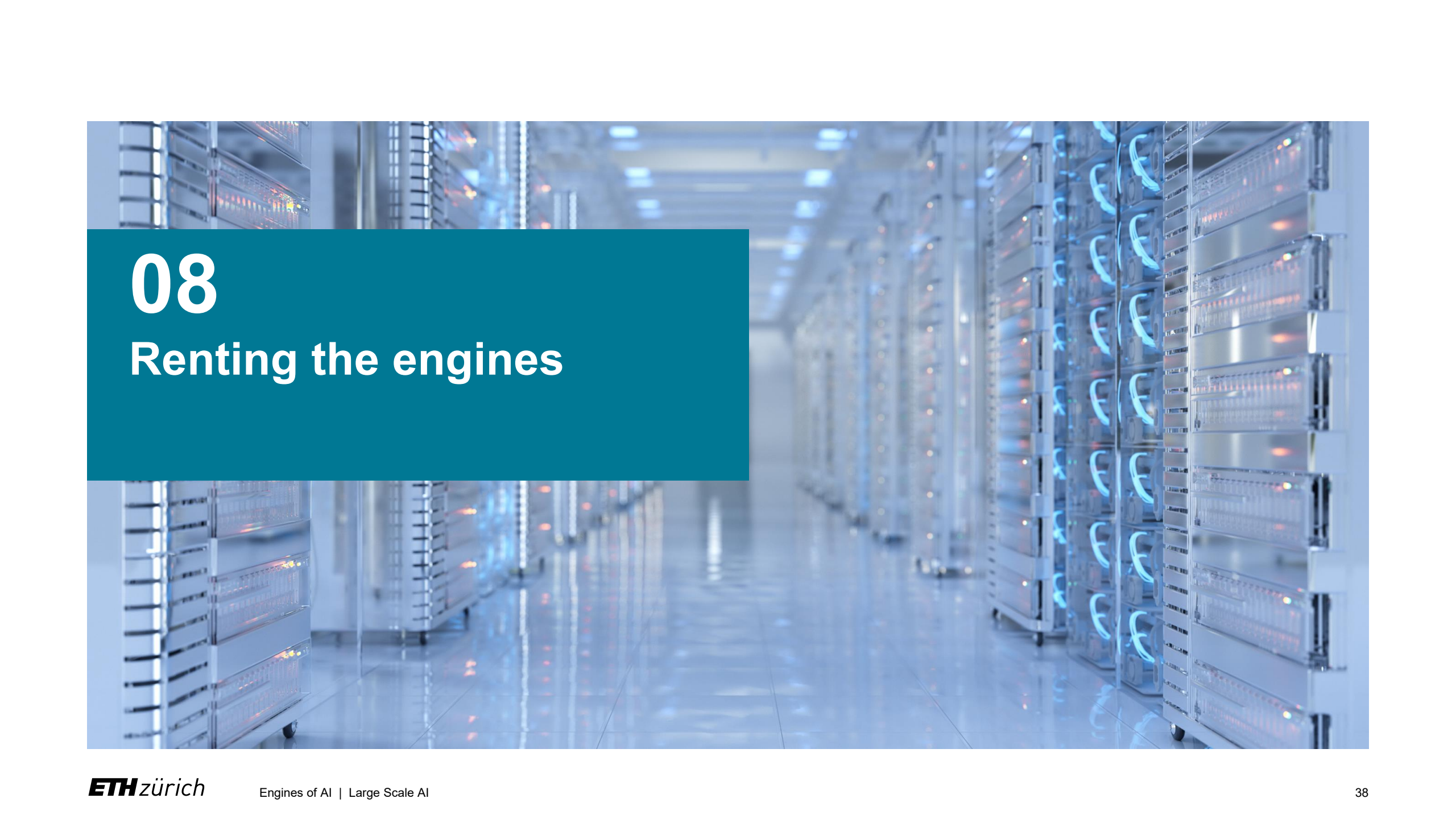
ASIC — an Application-Specific Integrated Circuit. Give up flexibility, gain enormous efficiency.

The systolic array: arithmetic on a conveyor belt



CPU versus GPU versus TPU

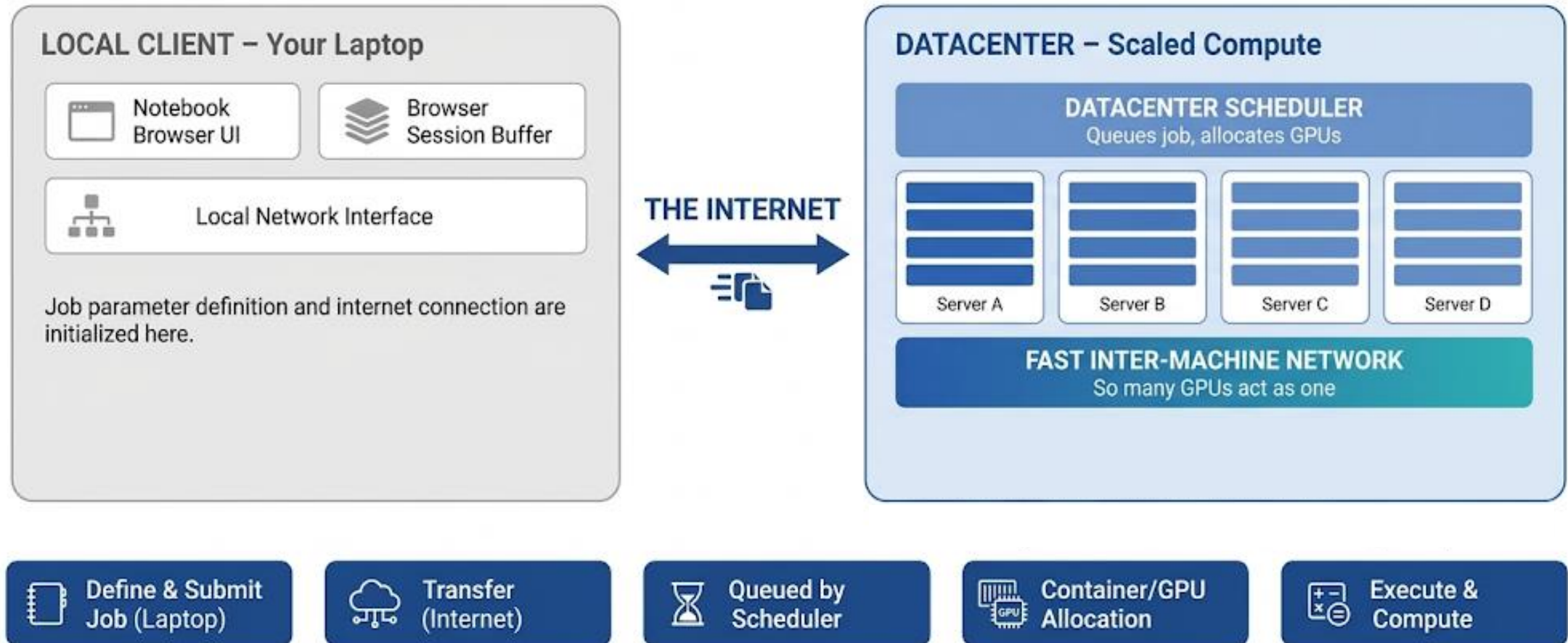
	CPU	GPU	TPU
Processors	4 – 16 general cores	Thousands of simple cores	A few very large matrix units
Flexibility	Runs anything	Runs parallel maths well	Runs neural-network maths only
Best at	Logic, control, sequence	Training, graphics, science	Large-scale training and serving
Compute			
Picture it as	The head chef: can prep, cook, customize	Fast-food crew: Hundred line cooks output perfect burgers simultaneously on hearing “flip”.	Packaged food: ingredients in, production line, package out. One product only.



08

Renting the engines

How you rent a GPU you will never see



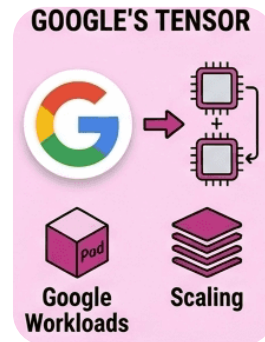
If AI needs an entire datacentre ... how is it running on your phone?

Face unlock. Live translation. Tidying up your photos. No internet required.

The NPU: the TPU's cousin, in your pocket

TPU — in a datacentre

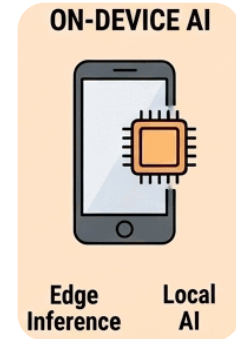
- Hundreds of watts.
- Trains models across thousands of chips.
- Lives in a rack, cooled by water.



same maths
 $\approx 1000\times$ less power

NPU — in your device

- Roughly one watt.
- Runs models that are already trained.
- Lives inside your phone's main chip.



Apple calls theirs the Neural Engine — in every iPhone since 2017.
Google has Tensor, Qualcomm has Hexagon, and laptops now ship with NPUs too.

Training in the datacentre. Answering, mostly, on the device.

What we covered

1

A CPU has a few very clever cores and works through things in a sequence.

2

Drawing pictures is matrix multiplication, millions of times over.

3

A GPU has thousands of simple cores doing exactly that, in parallel.

4

Neural networks turned out to need the very same arithmetic.

5

GPU hardware is important for training AI models, but so are the software layers that enable it — that's how NVIDIA won on CUDA.

6

A whole hardware supply chain rose with it: ASML, TSMC, the memory makers.

7

The TPU trades flexibility for efficiency — an Application-Specific Integrated Circuit

8

Your data travels over the internet for training to happen in the data centre GPUs.

9

Most edge devices now ship with NPU to enable real-time inferencing of models.

Thank You